L30 — Graphs: Terminology, Adjacency Matrix, Path Matrix

Unit: 3 | **Chapter:** Graphs | **BT Level:** BT3 | **CO:** CO2

Learning Objectives

- Define graph terminology: vertices, edges, degree, path, cycle
- Distinguish directed vs undirected, weighted vs unweighted graphs
- Represent graphs using adjacency matrix and adjacency list
- Compute the path matrix using matrix multiplication

---

1. Graph Fundamentals

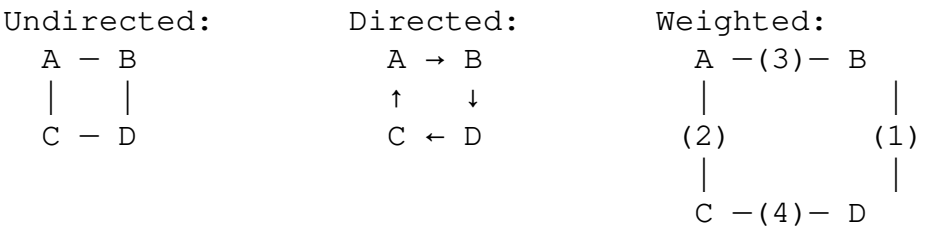
A **graph** G = (V, E) consists of:

- **V** — a set of **vertices** (nodes)
- **E** — a set of **edges** (connections between vertices)

1.1 Key Terminology

Term	Definition
Vertex (node)	Basic unit of a graph
Edge (arc)	Connection between two vertices
Adjacent	Two vertices connected by an edge
Degree	Number of edges incident to a vertex
In-degree	(Directed) Number of edges pointing **into** a vertex
Out-degree	(Directed) Number of edges pointing **out of** a vertex
Path	Sequence of vertices connected by edges
Simple path	Path with no repeated vertices
Cycle	Path that starts and ends at same vertex
Connected	There exists a path between every pair of vertices
Weighted graph	Each edge has a numerical weight
Directed graph (digraph)	Edges have a direction (u → v ≠ v → u)

1.2 Types of Graphs



1.3 Special Graphs

Type	Property
Simple graph	No loops or multiple edges
Complete graph K_n	Every pair of vertices is connected; n(n-1)/2 edges
Bipartite	Vertices split into 2 groups; edges only between groups
Tree	Connected graph with no cycles; n-1 edges for n vertices
DAG	Directed Acyclic Graph — no directed cycles

---

2. Graph Representation

2.1 Adjacency Matrix

An $n \times n$ matrix A where $A[i][j] = 1$ if edge (i,j) exists, else 0.

Undirected graph:

```
     A  B  C  D
A  [ 0  1  1  0 ]
B  [ 1  0  1  1 ]
C  [ 1  1  0  1 ]
D  [ 0  1  1  0 ]
```

Properties:

- Symmetric for undirected graphs ($A = A^T$)
- Space: $O(V^2)$
- Check edge (u,v): O(1)
- Find all neighbors of u: O(V)

For weighted graphs: $A[i][j] =$ weight (or ∞ if no edge)

```
def build_adjacency_matrix(vertices, edges, directed=False):
    n = len(vertices)
    idx = {v: i for i, v in enumerate(vertices)}
    matrix = [[0] * n for _ in range(n)]

    for u, v in edges:
        matrix[idx[u]][idx[v]] = 1
        if not directed:
            matrix[idx[v]][idx[u]] = 1

    return matrix, idx

# Example
vertices = ['A', 'B', 'C', 'D']
edges = [('A', 'B'), ('A', 'C'), ('B', 'C'), ('B', 'D'), ('C', 'D')]
matrix, idx = build_adjacency_matrix(vertices, edges)

for row in matrix:
    print(row)
```

2.2 Adjacency List

Each vertex stores a list of its neighbors.

```
from collections import defaultdict

def build_adjacency_list(vertices, edges, directed=False):
    graph = defaultdict(list)

    for u, v in edges:
        graph[u].append(v)
        if not directed:
            graph[v].append(u)

    return dict(graph)

graph = build_adjacency_list(
    ['A', 'B', 'C', 'D'],
    [('A', 'B'), ('A', 'C'), ('B', 'C'), ('B', 'D'), ('C', 'D')]
)
# {'A': ['B', 'C'], 'B': ['A', 'C', 'D'], 'C': ['A', 'B', 'D'], 'D': ['
```

Properties:

- Space: O(V + E)
- Find all neighbors of u: O(degree(u))
- Check if edge (u,v) exists: O(degree(u))

2.3 Comparison

Feature	Adjacency Matrix	Adjacency List
Space	$O(V^2)$	O(V + E)
Edge check	O(1)	O(degree)
Find neighbors	O(V)	O(degree)
Add edge	O(1)	O(1) amortized
Best for	Dense graphs	Sparse graphs

---

3. Path Matrix (Transitive Closure)

The **path matrix** P[i][j] = 1 if there is a path from vertex i to vertex j (possibly multi-hop).

3.1 Using Matrix Multiplication

If A is the adjacency matrix, then A^k[i][j] counts paths of exactly k hops from i to j.

The **reachability matrix** (Boolean path matrix) can be computed by:

$$P = A \lor A^2 \lor A^3 \lor \ldots \lor A^{n-1}$$

where $\lor$ is element-wise OR (or sum > 0).

```
def matrix_multiply_bool(A, B, n):
    """Boolean matrix multiplication: C[i][j] = OR of (A[i][k] AND B[k]
    C = [[0]*n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                if A[i][k] and B[k][j]:
                    C[i][j] = 1
    return C

def path_matrix_naive(adj, n):
    """Compute path matrix via repeated matrix multiplication."""
    # P = A | A² | A³ | ... | A^(n-1)
    current = [row[:] for row in adj]   # Copy of A
    path = [row[:] for row in adj]      # Starts as A

    for _ in range(n - 2):   # Multiply n-2 more times
        current = matrix_multiply_bool(current, adj, n)
        for i in range(n):
            for j in range(n):
                path[i][j] |= current[i][j]

    # Add identity (i to i always reachable)
    for i in range(n):
        path[i][i] = 1

    return path
```

3.2 Warshall's Algorithm ($O(V^3)$)

More efficient: compute transitive closure directly.

```
def warshall(adj, n):
    """
    Warshall's algorithm for transitive closure (path matrix).
    Modifies adj in place.
    """
    P = [row[:] for row in adj]
    for i in range(n):
        P[i][i] = 1   # Every vertex is reachable from itself

    for k in range(n):         # Intermediate vertex
        for i in range(n):     # Source
            for j in range(n): # Destination
                P[i][j] = P[i][j] or (P[i][k] and P[k][j])

    return P
```

Interpretation: `P[i][j] = 1` iff there exists a path from i to j.

---

4. Degree Analysis

```
def degree_analysis(adj_list, directed=False):
    if not directed:
        degrees = {v: len(neighbors) for v, neighbors in adj_list.items
        print("Degrees:", degrees)
        print("Handshake lemma: sum =", sum(degrees.values()), "(should
    else:
        out_degree = {v: len(neighbors) for v, neighbors in adj_list.it
        all_vertices = set(adj_list.keys())
        in_degree = {v: 0 for v in all_vertices}
        for v, neighbors in adj_list.items():
            for u in neighbors:
                in_degree[u] += 1
        print("Out-degrees:", out_degree)
        print("In-degrees:", in_degree)
```

Handshake Lemma: In an undirected graph, sum of all degrees $= 2|E|$.

Summary

- Graph $G = (V, E)$; edges can be directed/undirected, weighted/unweighted.
- **Adjacency matrix:** $O(V^2)$ space, $O(1)$ edge lookup — best for dense graphs.
- **Adjacency list:** $O(V + E)$ space — best for sparse graphs.
- **Path matrix:** `P[i][j]=1` iff path exists from i to j; computed via Warshall's algorithm in $O(V^3)$.

References

- Cormen et al., *Introduction to Algorithms*, Ch. 22 (MIT Press, 2022)
- Skiena, *The Algorithm Design Manual*, Ch. 5 (Springer, 2020)